DAA LAB ASSIGNMENT WEEK-1

NAME: KURRA MUNI SANTHAN KUMAR REDDY

ROLL.NO: CH.SC.U4CSE24225

1)SUM OF FIRST N NATURAL NUMBERS

CODE:

```
#include <stdio.h>

int sum_n_numbers(int n){
   int sum = 0;
   for(int i = 0; i <= n; i++){
      sum += i;
   }
   return sum;
}

int main(){
   int n;
   printf("Enter any natural number: ");
   scanf("%d",&n);
   printf("The sum of %d natural numbers is %d",n,sum_n_numbers(n));
   return 0;
}
```

OUTPUT:

```
santhan@Santhan-PC:~$ ./sumofnnaturalnumbers
Enter any natural number: 9
The sum of 9 natural numbers is 45santhan@Santhan-PC:~$
```

Interpretation: I declared a variable sum initialized to 0. Then I used a for loop from 1 to n adding i to sum each iteration. This directly computes the desired total.

Time complexity: O(n) — single loop over n elements.
Space complexity: O(1).

2)SUM OF SQUARES OF FIRST N NATURAL NUMBERS

CODE:

```
#include <stdio.h>


int sum_squares_n_numbers(int n){
   int sum = 0;
   for(int i = 0; i <= n; i++){
      sum += (i*i);
   }
   return sum;
}


int main(){
   int n;
   printf("Enter any natural number: ");
   scanf("%d",&n);
```

```
    printf("The sum of squares %d natural numbers is
%d",n,sum_squares_n_numbers(n));

    return 0;

}
```

OUTPUT:

```
santhan@Santhan-PC:~$ ./sumofsquaresoffirstnnaturalnumbers
Enter any natural number: 11
The sum of squares 11 natural numbers is 506santhan@Santhan-PC:~$
```

Interpretation: Similar to the previous problem, but this time I used a loop and added i*i each iteration into sum.

Time complexity: O(n).

Space complexity: O(1).

3)SUM OF CUBES OF FIRST N NATURAL NUMBERS

CODE:

```
#include <stdio.h>


int sum_cubes_n_numbers(int n){

    int sum = 0;

    for(int i = 0; i <= n; i++){

        sum += (i*i*i);

    }

    return sum;

}
```

```
int main(){
   int n;
   printf("Enter any natural number: ");
   scanf("%d",&n);
   printf("The sum of cubes of %d natural numbers is
%d",n,sum_cubes_n_numbers(n));
   return 0;
}
```

OUTPUT:

```
santhan@Santhan-PC:~$ ./sumofcubesoffirstnnaturalnumbers
Enter any natural number: 14
The sum of cubes of 14 natural numbers is 11025santhan@Santhan-PC:~$
```

Interpretation: Same approach as previous problems but adding i*i*i.

Time complexity : O(n).

Space complexity: O(1).

4)FACTORIAL OF A NUMBER USING RECURSION

CODE:

```
#include <stdio.h>


int factorial(int n){
   if(n < 0){
      return -1;
   }
   if(n == 0 || n == 1){
      return 1;
```

```
    }
    return n * factorial(n-1);
}

int main(){
    int n;
    printf("Enter a number: ");
    scanf("%d",&n);
    printf("The factorial of %d is %d",n,factorial(n));
    return 0;
}
```

OUTPUT:

```
santhan@Santhan-PC:~$ ./factorialofanumberusingresursion
Enter a number: 7
The factorial of 7 is 5040santhan@Santhan-PC:~$
```

Interpretation: The recursive function factorial returns 1 for n <= 1 (base case) and otherwise returns n * factorial(n-1).

Time complexity: O(n)

Space complexity: O(n)

5)TRANSPOSE OF A MATRIX

CODE:

```
#include <stdio.h>

int main(){
   int rows;
   int cols;
   printf("Enter the size of the matrix\n");
   printf("Enter number of rows: ");
   scanf("%d",&rows);
   printf("Enter number of columns: ");
   scanf("%d",&cols);

   int matrix[rows][cols];
   printf("Enter the values of the matrix: \n");
   for(int i = 0; i < rows; i++){
      for(int j = 0; j < cols; j++){
         printf("Enter the value at %d row and %d col: ",i+1,j+1);
         scanf("%d",&matrix[i][j]);
      }
   }

   int transMatrix[cols][rows];
   printf("The matrix is: \n");
   for(int i = 0; i < rows; i++){
      for(int j = 0; j < cols; j++){
```

```
            printf("%d\t",matrix[i][j]);
        }
        printf("\n");
    }

    for(int i = 0; i < cols; i++){
        for(int j = 0; j < rows; j++){
            transMatrix[i][j] = matrix[j][i];
        }
    }

    printf("The matrix after transpose is: \n");
    for(int i = 0; i < cols; i++){
        for(int j = 0; j < rows; j++){
            printf("%d\t",transMatrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

OUTPUT:

```
santhan@Santhan-PC:~$ ./transposeofamatrix
Enter the size of the matrix
Enter number of rows: 3
Enter number of columns: 2
Enter the values of the matrix:
Enter the value at 1 row and 1 col: 1
Enter the value at 1 row and 2 col: 2
Enter the value at 2 row and 1 col: 3
Enter the value at 2 row and 2 col: 4
Enter the value at 3 row and 1 col: 5
Enter the value at 3 row and 2 col: 6
The matrix is:
1       2
3       4
5       6
The matrix after transpose is:
1       3       5
2       4       6
```

Interpretation: First we read the number oof rows and colums for the matrix and also read the values for it, and then create another matrix with the opposite dimensions, and add values to it as a[i][j] -> b[j][i] using nested loops

Time Complexity: O(m*n).* Where m = no of rows, n= no of columns

Space Complexity: O(m*n).*

6)FIBONACCI SERIES UPTO A GIVEN NUMBER

CODE:

```
#include <stdio.h>

void fibonacci(int n){
   int curr = 1;
   int prev = 0;
   int i = 0;

   if(n < 0){
      printf("Enter a valid number!!");
      return;
   }

   while(i <= n){
      printf("%d ",curr);
      int temp = curr;
      curr = curr + prev;
      prev = temp;
      i++;
   }
   return;
}

int main(){
   int n;
```

```
    printf("Enter any number: ");
    scanf("%d",&n);
    fibonacci(n);
    return 0;
}
```

OUTPUT:

```
santhan@Santhan-PC:~$ ./fibonacciseries
Enter any number: 10
1 1 2 3 5 8 13 21 34 55 89 santhan@Santhan-PC:~$
```

Interpretation: Start with prev = 0, and curr = 1 when i=0, and print the number while updating the previous and current variables as curr = curr + prev, and prev = curr.

Time Complexity: O(n) **Space Complexity:** O(1)